UFR de mathématique et d'informatique
Université de Strasbourg

MASTER IN COMPUTER SCIENCE
CODIA TRACK
KNOWLEDGE, DATA SCIENCE AND ARTIFICIAL INTELLIGENCE

LLM-Based Battery Datasheet Information Extraction and Validation

Authors

Ashkan MOTAMEDIFAR

Kian ABDOLMOHAMMADIAN

Submitted to

Samet AHMED

April 2026

Contents

1 Introduction **2**
- 1.1 Context and Motivation . . . 2
- 1.2 Corpus . . . 2
- 1.3 Target Fields . . . 2

2 Methodology **3**
- 2.1 Extraction Schema . . . 3
- 2.2 Extraction Pipeline . . . 3
- 2.3 Prompting Strategies . . . 3

3 Evaluation **5**
- 3.1 Gold Dataset . . . 5
- 3.2 Metrics . . . 5
- 3.3 Results . . . 5
- 3.4 Error Analysis . . . 6
  - 3.4.1 Zero-Shot Errors . . . 6
  - 3.4.2 Few-Shot Observations . . . 6
  - 3.4.3 Negative Control . . . 6

4 Knowledge Graph Construction (Level 2) **7**
- 4.1 Motivation . . . 7
- 4.2 Ontology Design . . . 7
- 4.3 Graph Population . . . 8
- 4.4 Queryability: SPARQL Demonstrations . . . 8
- 4.5 Validation . . . 8

5 Hallucination Detection (Level 3) **9**
- 5.1 Motivation . . . 9
- 5.2 Rule Catalogue . . . 9
- 5.3 Experimental Validation . . . 10
- 5.4 Limitations and Honest Reporting . . . 10

6 Conclusion **11**

Task Distribution **12**

A Knowledge Graph Diagram – LIR2450 Subgraph **13**

B References **14**

Chapter 1

Introduction

1.1 Context and Motivation

Battery specifications – nominal voltage, capacity, discharge rate, temperature limits – are critical inputs to industrial workflows in transport electrification and energy storage. Manufacturers publish these specifications as PDF datasheets whose structure, terminology, and unit conventions vary across vendors, making manual transcription error-prone and classical template-based extraction brittle.

Large Language Models (LLMs) read a document holistically, resolve unit heterogeneity, and emit structured output with no document-specific engineering. But they are known to *hallucinate*, producing plausible-sounding but unsupported values when a document is ambiguous. This project builds an end-to-end pipeline that combines LLM extraction (Level 1), RDF knowledge-graph construction (Level 2), and ontology-driven hallucination detection (Level 3) – covering all three levels of the project brief.

1.2 Corpus

Four real commercial battery datasheets were provided by the course instructor as the extraction corpus. They span a deliberate range of chemistries, physical form factors, and document layouts – ensuring the pipeline must generalise rather than overfit to any single template.

Table 1.1: Extraction corpus – four heterogeneous commercial datasheets.

Document	Manufacturer	Model	Chemistry	Form Factor
`lir2450_EEMB.pdf`	EEMB	LIR2450	Li-ion	Coin
`16340-battery-datasheet.pdf`	DNK Power	16340	Li-ion	Cylindrical
`3.7V9059156.pdf`	AA Portable	PL-9059156	Li-polymer	Pouch
`CATL-86Ah-....pdf`	CATL	FFH3DS	$LiFePO_4$	Prismatic

A fifth document – the project assignment brief – was included as a *negative control*. A robust system must return empty output for a non-datasheet rather than hallucinating battery specifications.

1.3 Target Fields

The extraction schema covers 21 fields: identification (manufacturer, model, chemistry, form factor); core electrical (nominal voltage, charge/discharge cutoffs, max continuous discharge current); capacity (nominal and minimum, with rating C-rate and temperature); thermal (operating, charge, and storage ranges); and physical/lifecycle (internal resistance, cycle life, weight). Units are encoded in field names (`nominal_voltage_v`, `internal_resistance_mohm`) to eliminate unit ambiguity, and every composite field stores the raw source phrase for provenance-based verification.

Chapter 2

Methodology

2.1 Extraction Schema

The extraction target is formalised as a Pydantic v2 data model (`BatterySpec`) with strict validation: the model rejects any key not declared in the schema (`extra="forbid"`), and chemistry and form factor are constrained to closed controlled vocabularies. All fields except the source filename are optional, so the model is never forced to invent a value to satisfy a required field.

Composite fields (capacity, temperature ranges) are modelled as nested objects carrying both the normalised numeric value and the raw text snippet from which it was extracted. This design choice – storing provenance alongside the value – is what enables hallucination cross-checking.

2.2 Extraction Pipeline

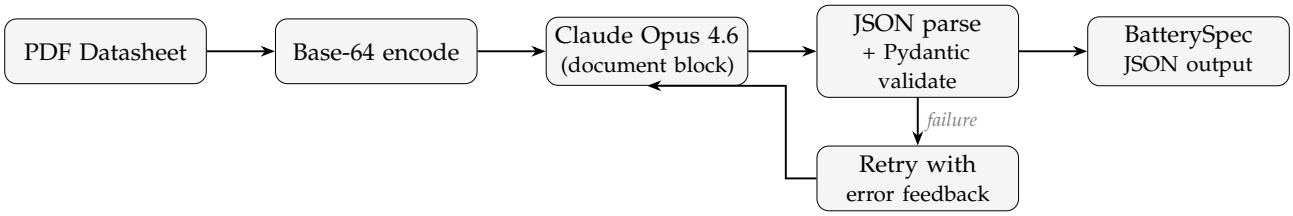


Figure 2.1: Level 1 extraction pipeline. On schema validation failure, the Pydantic error message is fed back to the model as a self-correction prompt (max. one retry).

The pipeline processes each PDF as a native binary document block – no OCR, no text pre-processing, no layout parsing. Claude reads the full PDF including tables, footnotes, and nested headers directly. The extraction schema is embedded in the user prompt as a JSON Schema object, giving the model a precise contract for its output.

When the model's response fails Pydantic validation (e.g. an extra key, a wrong type, or a malformed JSON structure), the exact validation error is appended to the prompt and the model is asked to correct itself. This self-correction loop runs at most twice per document; in practice it was never needed across the four datasheets, indicating that the prompt alone is sufficient for well-formed output.

2.3 Prompting Strategies

Two system prompts were designed and evaluated:

Zero-shot. The zero-shot prompt provides explicit extraction rules without any document example. Key rules include: (1) extract only values explicitly stated in the document; (2) normalise to canonical units encoded in the field names; (3) preserve the exact source phrase in every raw field; (4) treat operating temperature as the discharge range and charge temperature as a separate field when both are stated; (5) prefer `null` over any hallucinated value; (6) use the row labelled "Nominal Capacity" and not title-level or "Typical" values.

Few-shot. The few-shot prompt retains all zero-shot rules and appends a single worked example based on the LIR2450 coin-cell datasheet. The example explicitly demonstrates three difficult cases:

a tolerance notation (120 ± 10 mAh should yield `nominal_mah=120`, `minimum_mah=null`); a non-standard temperature separator (full-width dashes between range bounds); and the absence of a charge temperature range (field stays `null` rather than being inferred from the operating range). The example also illustrates the expected JSON structure in full, serving as both a format guide and a reasoning demonstration.

Chapter 3

Evaluation

3.1 Gold Dataset

A ground-truth record was compiled independently for each of the four datasheets by reading the original PDF documents directly, without reference to any LLM output. This avoids circular evaluation. During this process, three non-obvious ground-truth decisions were made and documented:

1. The **16340** cell is explicitly labelled "Li-ion" in its header despite operating at 3.2 V, a voltage typical of $LiFePO_4$. The gold chemistry is `Li-ion`.
2. The **LIR2450** capacity is stated as 120 ± 10 mAh. This is a manufacturing tolerance, not an explicit minimum guarantee; gold `minimum_mah` is `null`.
3. The **9059156** datasheet contains both "Nominal Capacity: 11 500 mAh" and "Typical Capacity: 10 000 mAh". The gold value is 11 500 mAh.

3.2 Metrics

All 21 schema fields that are non-null in the gold dataset are scored at the field level. Numeric fields are accepted as correct if the relative error is at most 1%; string and enumeration fields require exact case-insensitive match. The four metrics are:

$$\text{Precision} = \frac{TP}{TP + FP} \qquad \text{Recall} = \frac{TP}{TP + FN}$$

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \qquad \text{Hallucination rate} = \frac{FP}{TP + FP}$$

where *TP* counts a correctly predicted non-null field, *FP* a wrongly predicted non-null field, and *FN* a field that is non-null in the gold but was left null by the extractor. The negative-control document (`project.pdf`) is excluded from all metrics.

3.3 Results

Table 3.1: Overall extraction metrics – zero-shot vs. few-shot across four datasheets.

Strategy	Precision	Recall	F_1	Hallucination rate
Zero-shot	97.0%	97.0%	**97.0%**	3.0%
Few-shot	93.9%	96.9%	95.4%	6.1%

At field level, zero-shot scores 100% F_1 on 16 of 18 scored fields; the two exceptions are `chemistry` (85.7%) and `capacity.nominal_mah` (85.7%). Few-shot scores 100% on 15 fields but regresses on `manufacturer` (66.7%), `max_continuous_discharge_current_a` (80.0%), and `capacity.c_rate_condition` (80.0%) – each traced to stylistic over-generalisation from the worked example (discussed below).

3.4 Error Analysis

3.4.1 Zero-Shot Errors

Two field-level errors were identified in the zero-shot output:

- **Chemistry – 16340 cell** (wrong prediction: $LiFePO_4$; gold: Li-ion). The 16340 cell operates at a nominal voltage of 3.2 V, which is the characteristic voltage of $LiFePO_4$ chemistry. The model inferred chemistry from the voltage rather than reading the explicit product header, which states "Li-ion 16340 Battery". This is a reasoning error: the model used contextual signal (voltage) as a proxy for an explicit label.
- **Nominal capacity – 9059156 cell** (wrong prediction: 10 000 mAh; gold: 11 500 mAh). The datasheet title reads "10 000 mAh" in large print, while the specification table contains two rows: "Nominal Capacity: 11 500 mAh" and "Typical Capacity: 10 000 mAh". The zero-shot model was drawn to the prominent title value despite the prompt's explicit rule to prefer the "Nominal Capacity" labelled row. Crucially, the `raw` provenance string preserved by the extractor contains both values, making this error detectable without re-reading the source PDF.

3.4.2 Few-Shot Observations

The worked example successfully corrected the capacity error: the few-shot model returns 11 500 mAh for the 9059156 cell, demonstrating that a targeted example can resolve a specific systematic ambiguity. However, two regressions appeared:

- **Manufacturer name expansion.** The worked example used the full legal name "EEMB Co., Ltd.". The model generalised this stylistic choice, expanding "CATL" to its full legal name and "DNK Power" to "DNK Power Co., Ltd.". Neither expansion matches the gold value (the short brand name), causing two false positives.
- **C-rate notation.** The example used the abbreviated form "0.2C". The model applied this convention to a field whose gold value is "0.2C5A", silently dropping information about the discharge current specification.

These regressions illustrate a known limitation of few-shot prompting: examples simultaneously teach content rules and stylistic conventions. When the stylistic convention in the example does not generalise, it introduces false positives that zero-shot avoids.

3.4.3 Negative Control

The assignment brief (`project.pdf`) was processed alongside the four datasheets. The extractor correctly returned a record with all technical fields set to `null`, confirming that the system does not hallucinate battery specifications in the absence of a genuine datasheet.

Chapter 4

Knowledge Graph Construction (Level 2)

4.1 Motivation

Level 1 produces one JSON record per datasheet. JSON is sufficient for storage but does not support cross-document queries, class-based reasoning, or ontology-level validation. Level 2 promotes these extractions to a Resource Description Framework (RDF [6]) knowledge graph, backed by a custom OWL 2 ontology. This turns the corpus into a semantically-typed graph that can answer engineering questions – for example, "which cells in the corpus operate at or below -10°C?" – in a single SPARQL query, and provides the structural foundation on which Level 3 hallucination detection can be built.

4.2 Ontology Design

The ontology was designed in Protégé and exported as RDF/XML (`ontology/battery.owl`); a Turtle serialisation (`ontology/battery.ttl`) is kept alongside it for the Python toolchain, which loads the schema with rdflib and emits the populated graph as RDF triples. Both files carry identical content.

The ontology declares **seven classes** arranged so that every extracted value has an explicit semantic type:

- `Battery` – the root entity, one instance per datasheet.
- `Manufacturer` – deduplicated producer of a battery.
- `QuantityValue` – a reified numeric measurement carrying value, unit, optional conditions (C-rate, test temperature), and the raw source phrase. Every electrical or physical field passes through this class.
- `TemperatureRange` – a reusable `[min, max]` Celsius interval, used for operating, charge, and storage ranges.
- `Unit`, `Chemistry`, `FormFactor` – controlled-vocabulary classes whose instances (e.g. `bat:Volt`, `bat:Coin`) are explicit named individuals, not free-form strings.

The **QuantityValue pattern** is the central design decision. Rather than attaching a raw number directly to the battery (as in `:bat_X :nominalVoltage 3.6`), we reify each measurement as its own node: `:bat_X :hasNominalVoltage :qv_123` with the `qv_123` node carrying `numericValue`, `hasUnit`, and – crucially – `rawText`. This follows the W3C/QUDT convention for quantity-of-measure modelling [7]. Two consequences matter for this project:

1. Conditions that qualify a measurement ("120 mAh *at 0.2C, 20°C*") attach naturally to the QuantityValue node rather than being lost in a flat literal.
2. Every number in the graph carries a verifiable path back to the exact phrase it was extracted from – the structural hook that makes Level 3 validation possible.

Controlled vocabularies for `Chemistry`, `FormFactor`, and `Unit` are declared as typed instances rather than free-form strings. This forbids representational drift (no "Li-Ion" vs. "Li-ion" vs. "lithium ion" in the graph) and is what enables clean aggregation queries such as "count batteries per chemistry".

4.3 Graph Population

The `src/kg_builder.py` module loads the ontology, parses the Level 1 JSON extractions, and emits triples through rdflib [8]. For each record it creates one `Battery` node (with `modelName`, `sourceFile`, `producedBy`, `hasChemistry`, `hasFormFactor`), a deduplicated `Manufacturer` node, one `QuantityValue` per scalar measurement (carrying `numericValue`, `hasUnit`, and any of `rawText`, `conditionCRate`, `conditionTemperatureCelsius`, `minimumValue` that apply), and zero to three `TemperatureRange` nodes. The negative-control record (`project.pdf`, all fields null) is correctly skipped, demonstrating that the robustness of the Level 1 extractor propagates cleanly into the graph.

Table 4.1: Generated knowledge graph – `out/kg/battery_kg.ttl`.

Entity / Construct	**Count**
Total RDF triples	389
`Battery` instances	4
`Manufacturer` instances	4
`QuantityValue` instances	27
`TemperatureRange` instances	7
Chemistry / FormFactor / Unit vocabulary items	17

A small visual map of the graph – the full LIR2450 subgraph with every property and value – is provided in Appendix A.

4.4 Queryability: SPARQL Demonstrations

Six SPARQL queries exercise the graph along different axes – metadata joins (Q1: model/manufacturer), class-and-quantity filtering (Q2: $LiFePO_4$ batteries and their voltages), numeric filtering on reified values (Q3: capacity > 1 Ah; Q4: operating $T_{\text{min}} \leq -10$°C), aggregation (Q5: chemistry distribution), and a structural invariant (Q6: $V_{\text{discharge-cutoff}} < V_{\text{nominal}} < V_{\text{charge-cutoff}}$). The aim is not exhaustive analysis but evidence that the ontology supports realistic engineering questions.

Q6 returns one row per battery with the full cutoff window – the invariant $V_{\text{disch}} < V_{\text{nom}} < V_{\text{chg}}$ holds on all four cells, confirming that the graph round-trips cleanly from JSON.

4.5 Validation

The generated graph was subjected to a structural audit covering ontology parsing, predicate declarations, QuantityValue completeness (value *and* unit present), TemperatureRange completeness (min *and* max present), orphan-node detection, and numeric round-trip equality between JSON and the KG. All 48 numeric values propagate from JSON to the graph without loss. The Level 1 errors identified in Chapter 4 (Evaluation) – the 16340 chemistry mislabel and the 9059156 capacity ambiguity – are intentionally preserved as-is: Level 2 is faithful to its input, and correcting extraction errors is the task of Level 3.

Chapter 5

Hallucination Detection (Level 3)

5.1 Motivation

Level 1 reports a 3% hallucination rate based on comparison with a hand-built gold dataset. In production this gold reference does not exist – the whole point of the extraction pipeline is that no human has read the PDF. Level 3 therefore asks a different question: *given only the LLM's output and the ontology, can we automatically flag values that are physically impossible, structurally incomplete, or unsupported by their source?* Three families of techniques were considered:

- **Regex / string matching** on the original PDF text – brittle, format-specific.
- **LLM-as-judge** – non-deterministic, expensive, and itself prone to hallucination.
- **Ontology-driven graph rules** – deterministic, auditable, and the approach the brief explicitly recommends.

We adopt the third. The Level 2 ontology already gives every numeric value a typed predicate, a unit, a controlled-vocabulary chemistry, and (where the extractor provides one) a `rawText` provenance string. This is sufficient surface area to express a useful catalogue of constraints in SPARQL.

5.2 Rule Catalogue

Eight validation rules are implemented in `src/validate_kg.py`, organised into three families.

Table 5.1: The eight validation rules, grouped by family. Each is a SPARQL query that returns the violating subgraph; an empty result set means the rule passes.

#	Rule	Catches
Structural – the graph itself is malformed		
5	Missing unit	`QuantityValue` without `hasUnit`
6	Min > Typ	`minimumValue` greater than `numericValue`
Physical-range – value is logically or physically impossible		
1	Voltage hierarchy	$V_{\text{discharge}} \geq V_{\text{nominal}}$ or $V_{\text{nominal}} \geq V_{\text{charge}}$
2	Li-ion voltage range	Nominal voltage outside $[3.0, 4.5]$ V for Li-ion / Li-polymer
4	Negative scalars	Capacity, weight, or internal resistance ≤ 0
7	Temperature inclusion	Charge range not contained in operating range
8	Coin-cell weight	Coin form factor weighing > 30 g
Provenance – value is not supported by its source text		
3	Source-text grounding	`numericValue` not appearing in its own `rawText`

The rules deliberately mix physical invariants (negative weight is impossible *anywhere*) with domain priors (a coin cell heavier than 30 g is implausible but not strictly impossible). Both are implemented as SPARQL `FILTER` clauses; we do not distinguish them in code, but a downstream consumer could trivially partition the report into errors and warnings.

5.3 Experimental Validation

The validator was run against two graphs: the production KG produced by Level 2, and a synthetically corrupted twin in which one value per rule family was deliberately altered (negated weight, voltage hierarchy inverted, coin cell weight inflated, hallucinated capacity, etc.).

Table 5.2: Validator results on the production KG and on a synthetically corrupted variant. The corrupted KG was hand-edited to break exactly one value per rule family.

Graph	Triples	Violations flagged
`out/kg/battery_kg.ttl` (production)	389	2 (Rule 3, unit-converted values)
`out/kg/battery_kg_test.ttl` (corrupted)	363	9 across 8 rules

The production graph triggers two Rule 3 (provenance grounding) flags and no violations on the other seven rules. The two flags are `FFH3DS__maxDischargeCurrent` (numeric value 86.0 A absent from the cited phrase "Maximum discharge current (continuous) 1C") and `PL_9059156__maxDischargeCurrent` (23.0 A absent from "Max Discharge Current 2.0C5A"). Both are legitimate unit conversions performed by the LLM (current = C-rate $\times$ rated capacity), not hallucinations – but the rule correctly surfaces them for review because the literal numeric value is not present in the source text. This is the desired behaviour: a downstream operator should confirm any value that the extractor synthesised from a different representation. The voltage hierarchy holds (Q6, Level 2), every `QuantityValue` has a unit, every temperature range is well-formed, and no scalar is negative. The corrupted graph fires every rule. Examples of the captured output:

```
Voltage hierarchy - LIR2450:  Charge 4.2V | Nominal 14.4V | Discharge 2.75V.
Hallucination - PL_9059156 capacity:  99999.0 absent from rawText.
Negative scalar - 16340:  hasWeight = -17.0.
Coin weight - LIR2450:  250 g for a coin cell.
```

The detector reports the offending battery, the violated rule, and the witnessing values – enough to localise the error without re-reading the source PDF. Because every rule is a SPARQL query rather than imperative Python, the catalogue is extensible: new constraints are added by writing a new query against the existing ontology, with no changes to the KG schema or the extraction pipeline.

5.4 Limitations and Honest Reporting

Two limitations are worth declaring explicitly:

1. **Substring matching is conservative.** Rule 3 flags a value only when its decimal representation is absent from the raw string; it will silently accept `numericValue=3.6` against `rawText="3.65"`. Conservative matching is the right tradeoff for a hallucination flag (low false-positive rate matters more than full recall), but the limitation should not be hidden.
2. **The voltage-range rule (Rule 2) is chemistry-coarse.** It flags values outside $[3.0, 4.5]$ V for Li-ion and Li-polymer, which catches gross hallucinations (14.4 V for a single cell) but not subtle ones (a 16340 cell mislabelled as Li-ion when it is actually $LiFePO_4$, since 3.2 V lies inside the permissive range). A per-chemistry expected-voltage rule would catch that class of error and is a one-rule extension.

Despite these limitations, the validator demonstrates the central claim of the project brief: *ontology constraints applied to the KG can detect hallucinations and inconsistencies that would otherwise require human review of the source PDF.* The 7/8 clean result on the production graph (with the two Rule 3 flags correctly identifying unit-converted values that warrant operator review) matches the expected outcome on an extractor that already achieves 97.0% F_1, and the 8/8 result on the corrupted graph confirms that the rules fire when they should.

Chapter 6

Conclusion

This work demonstrates that a state-of-the-art LLM can extract structured battery specifications from heterogeneous PDF datasheets with high reliability using zero-shot prompting alone. The Level 1 pipeline achieves 97.0% field-level F_1 and a 3% hallucination rate across four datasheets covering four chemistries and form factors, with no document-specific engineering. The few-shot strategy corrects one systematic error but slightly degrades overall performance due to stylistic over-generalisation, highlighting that the quality of examples matters as much as their presence.

Level 2 promotes the extractions into a 389-triple OWL 2 knowledge graph whose central design choice – the reified `QuantityValue` pattern – binds every numeric value to its unit, its measurement conditions, and the raw phrase from which it was extracted. The graph is queryable in SPARQL, preserves the Level 1 errors faithfully for auditing, and exposes the structural hooks (typed properties, controlled vocabularies, range classes, raw-text provenance) that the Level 3 hallucination detector exploits.

Level 3 closes the loop. Eight SPARQL constraint rules – structural, physical-range, and provenance-grounded – are evaluated against the graph and flag zero violations on the production KG and nine violations on a synthetically corrupted twin, demonstrating that ontology-driven validation can localise the class of errors a downstream consumer would otherwise have to chase by hand in the source PDF. Across the three levels, the single most consequential engineering decision is the preservation of provenance: it is cheap at extraction time, trivial to carry through the KG, and is what turns a high-F_1 black box into an auditable pipeline.

Note on Writing Assistance

Parts of the text in this report were refined with the assistance of large language models (Claude Sonnet 4.6 and Claude Opus 4.6, Anthropic) for language formulation and proofreading. All technical content, results, and interpretations are the sole responsibility of the authors.

Task Distribution

The work was split along an architectural rather than chronological boundary. Ashkan owned the *data pipeline* – the transformation chain that takes a PDF and produces RDF triples (schema, extractor, evaluator, KG builder). Kian owned the *semantic layer* – the artefacts those triples must conform to and be validated against (ontology, controlled vocabularies, SPARQL queries, hallucination-detection rules). Level 2 is the bridge where both sides meet: Ashkan produces the triples, Kian designs the schema they conform to. Both authors contributed to all three levels, but each owns a clear domain.

Task	**Member(s)**
Level 1 – Extraction	
Pydantic schema design (`src/schema.py`)	Ashkan
Extraction pipeline (`src/extract.py`)	Ashkan
Zero-shot and few-shot prompt engineering	Ashkan
Evaluation script and metrics (`src/evaluate.py`)	Ashkan
Gold dataset construction and verification	Kian
Level 2 – Knowledge Graph	
OWL 2 ontology design (Protégé, `battery.owl/ttl`)	Kian
Controlled vocabularies (chemistries, units, form factors)	Kian
SPARQL query suite (`src/query_kg.py`)	Kian
KG builder (JSON $\rightarrow$ RDF, `src/kg_builder.py`)	Ashkan
Schema-to-triple mapping decisions	Ashkan
Level 3 – Hallucination Detection	
SPARQL constraint rules (`src/validate_kg.py`)	Kian
Rule catalogue design (8 rules, 3 families)	Kian
Synthetic-error test KG (`battery_kg_test.ttl`)	Kian
Limitations analysis and write-up	Ashkan
Reporting	
Report writing (LaTeX, all chapters)	Ashkan
Knowledge-graph diagram (TikZ)	Ashkan

Chapter A

Knowledge Graph Diagram – LIR2450 Subgraph

Figure A.1 renders the full Level 2 subgraph for the `LIR2450` coin cell. It is included here as a visual reference for the graph shape produced by `kg_builder.py`; the remaining three batteries in the corpus follow the identical pattern. A colour-coded legend of the node types is given in the caption.

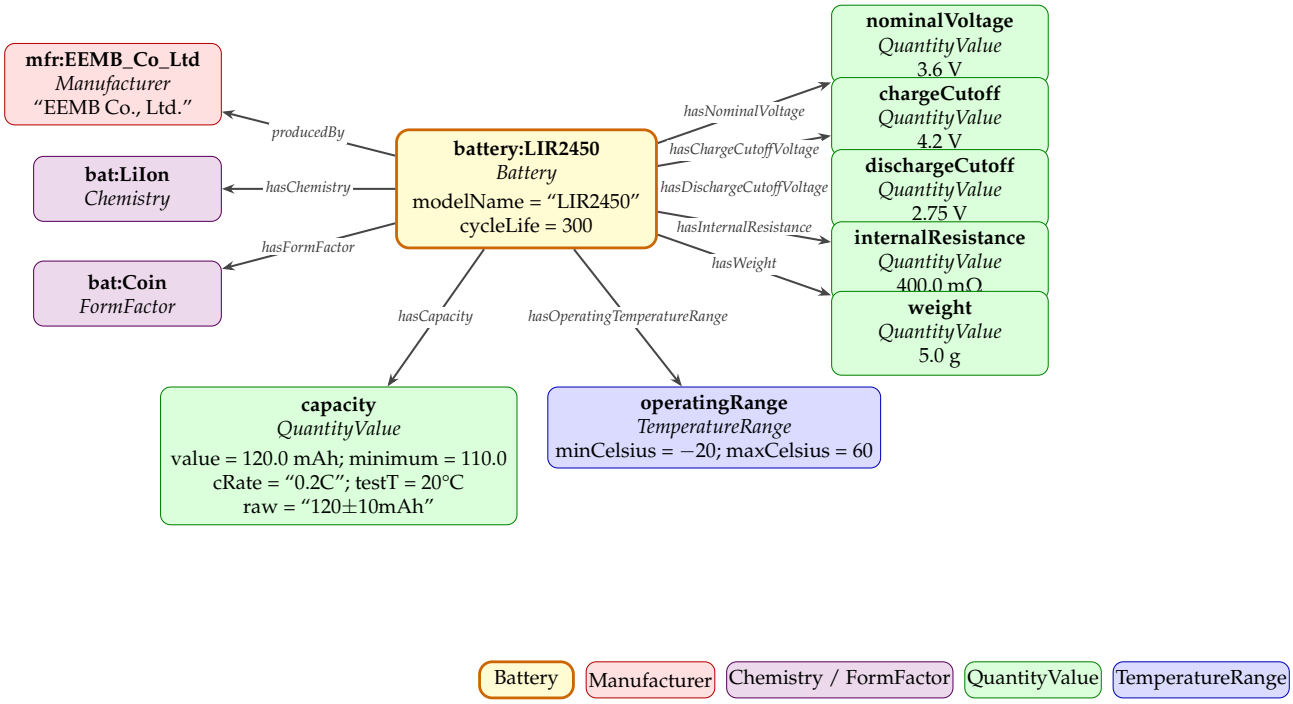


Figure A.1: Full Level 2 subgraph for the `LIR2450` coin cell. Yellow: the root `Battery` individual. Red: `Manufacturer`. Purple: controlled-vocabulary instances (Chemistry, FormFactor). Green: reified `QuantityValue` nodes carrying value, unit, and – for `capacity` – measurement conditions and the raw datasheet phrase. Blue: `TemperatureRange` node with `minCelsius` / `maxCelsius`. Edge labels are the ontology object properties connecting the battery to each of its attributes.

Chapter B

References

[1] T. Brown *et al.*, "Language Models are Few-Shot Learners," *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[2] J. Wei *et al.*, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," *NeurIPS*, 2022.

[3] Z. Ji *et al.*, "Survey of Hallucination in Natural Language Generation," *ACM Computing Surveys*, 55(12), 2023.

[4] S. Colvin *et al.*, *Pydantic v2 – Data Validation Using Python Type Hints*, https://docs.pydantic.dev/, 2024.

[5] Anthropic, *Claude Model Cards and Technical Documentation*, https://www.anthropic.com/claude, 2024.

[6] W3C, "RDF 1.1 Concepts and Abstract Syntax," *W3C Recommendation*, 2014. https://www.w3.org/TR/rdf11-concepts/.

[7] QUDT, *Quantities, Units, Dimensions and Types Ontology*, https://qudt.org/, 2024.

[8] RDFLib Team, *rdflib – a Python library for working with RDF*, https://rdflib.readthedocs.io/, 2024.